



Bases de Données Avancées PL/SQL

Mohammed ZOUITEN

Ecole Supérieure de Technologie
de Fès- Université Sidi mohammed
Ben Abdellah de Fès
mzouiten@gmail.com

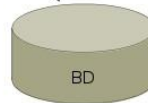
IL – Programmation de BD

Communication avec un SGBD utilisant SQL

Un énoncé en entrée

```
SQL> SELECT Prenom, Nom FROM POFESSEUR
```

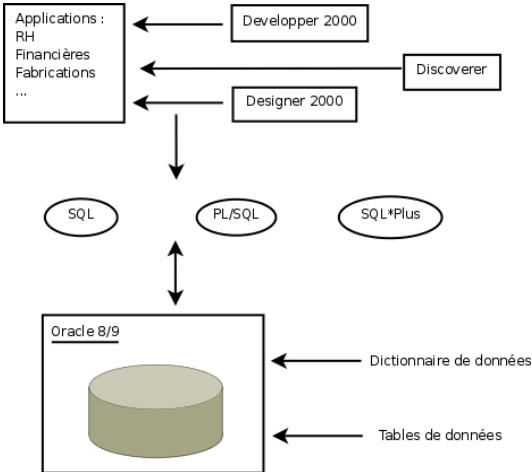
Envoi de l'énoncé à la BD



Affichage des données

Prenom	Nom
Brian	Kernighan
Dennis	Ritchie
Donald	Knuth
John	Backus
Edsger	Dijkstra
Marvin	Minski

Solution complète Oracle



Oracle (approche intégrée) :

```
sql

-- Tout dans la base
CREATE PROCEDURE traitement_complet AS
BEGIN
    -- SQL
    -- Logique métier
    -- Gestion erreurs
    -- Contrôle transactions
END;
```

Approche mixte (ex: Java + MySQL) :

```
java

// Java côté application
public void traitement() {
    // Connexion à la base
    Connection conn = DriverManager.getConnection(...);

    // Exécution SQL
    Statement stmt = conn.createStatement();
```

Clauses SQL

SELECT	Interrogation des données
INSERT UPDATE DELETE	Langage de Manipulation de Données (LMD)
CREATE ALTER DROP RENAME TRUNCATE	Langage de Définition de Données (LDD)
GRANT REVOKE	Langage de Contrôle de Données (LCD)
COMMIT ROLLBACK SAVEPOINT	Contrôle de transaction

```
TRUNCATE TABLE logs_application;
```

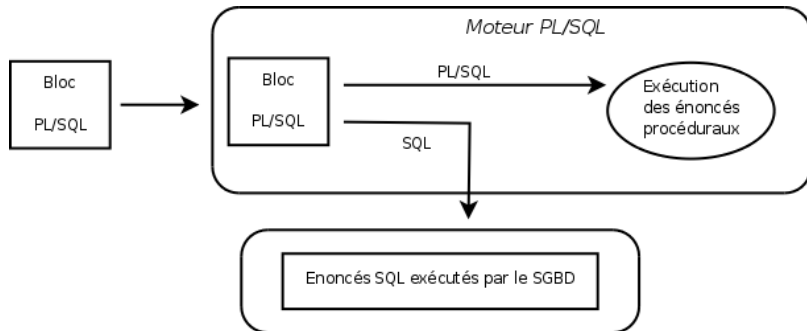
```
BEGIN;  
    SAVEPOINT niveau1;  
    -- Opérations...  
  
    SAVEPOINT niveau2;  
    -- Plus d'opérations...  
  
    -- Rollback à niveau1 annule aussi niveau2  
    ROLLBACK TO SAVEPOINT niveau1;  
COMMIT;
```

PL/SQL

Procedural Language for Structured Query Language

- Langage fournissant une interface procédurale au SGBD Oracle
- Intégration du langage SQL en lui apportant une dimension procédurale
- Réalisation de traitements algorithmiques (ce que ne permet pas SQL)
- Mise à disposition de la plupart des mécanismes classiques de programmation des langages hôtes tels que C, COBOL, PASCAL, C++, JAVA ...

Environnement PL/SQL



Avantages de PL/SQL

PL/SQL complément de SQL (qui n'est pas procédural)

Mécanismes offerts par PL/SQL :

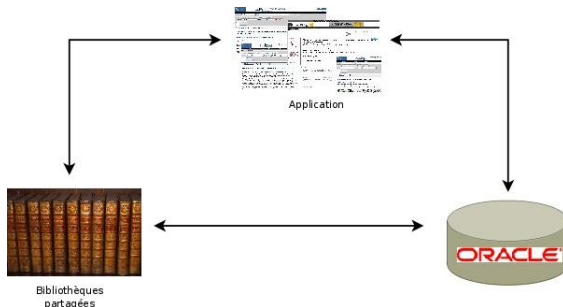
- Structures itératives : WHILE *** LOOP, FOR *** LOOP, LOOP ***
- Structures conditionnelles :
IF *** THEN *** ELSE | ELSEIF *** ENDIF, CASE ***
- Déclaration des curseurs et des tableaux
- Déclaration de variables
- Affectation de valeurs aux variables
- Branchements : GOTO, EXIT
- Exceptions : EXCEPTION

Avantages de PL/SQL

Intégration

- Meilleure cohérence du code avec les données
- Utilisation de librairies standards prédéfinies (*bibliothèques partagées*)

Ex : **DBMS_JOB / DBMS_SCHEDULER / DBMS_OUTPUT..**



Sans PL/SQL (SQL pur) :

```
sql

-- Envoi 1 au serveur
SELECT * FROM clients WHERE id = 100;

-- Envoi 2 au serveur
UPDATE commandes SET statut = 'PAYE' WHERE client_id = 100;

-- Envoi 3 au serveur
INSERT INTO logs VALUES ('Client 100 payé', SYSDATE);
```

→ 3 allers-retours réseau entre client et serveur

Avec PL/SQL (bloc SQL) :

```
sql

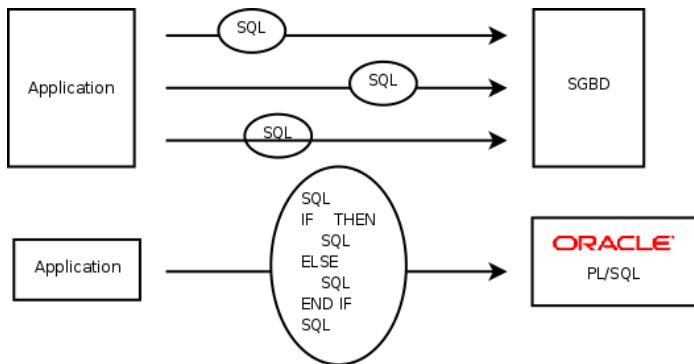
BEGIN
    -- Tout dans UN SEUL envoi
    SELECT * FROM clients WHERE id = 100;
    UPDATE commandes SET statut = 'PAYE' WHERE client_id = 100;
    INSERT INTO logs VALUES ('Client 100 payé', SYSDATE);
END;
```

→ 1 seul aller-retour réseau pour tout le traitement!

Avantages de PL/SQL

Blocs SQL

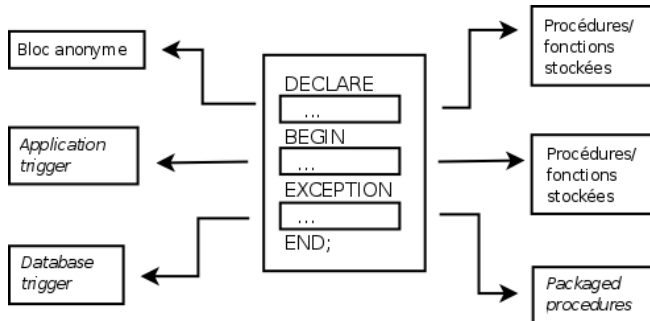
- Traitement par/de blocs SQL dans un énoncé PL/SQL (optimisation des transactions réseaux)



Avantages de PL/SQL

Exécution de programmes modulaires

- Traitements complexes (cas particuliers, erreurs)
- Traitements des exceptions
- ...



Avantages de PL/SQL

Résumé

- Langage portable
- Utilisation de variable de stockage
- Utilisation de type simple ou de type structuré dynamiquement (%TYPE, %ROWTYPE, etc.)
- Utilisation des structures de contrôle des langages procéduraux
- Gestion et manipulation des erreurs
- Création d'ordre SQL dynamique

Utilisation de PL/SQL

3 formes :

- Bloc de code, exécuté comme une commande SQL (utilisation d'un interpréteur standard SQL+ ou iSQL*PLus)
- Fichier de commandes PL/SQL
- Programme stocké : procédure, fonction, package ou trigger

Structure d'un bloc PL/SQL

-- *Section déclarative , optionnelle*

DECLARE

Variables , curseurs , exceptions , ...

-- *Section exécutable , obligatoire*

BEGIN

Instructions SQL et PL/SQL

Possibilités de blocs fils (imbrication de blocs)

-- *Section de traitement des exceptions , optionnelle*

EXCEPTION optionnelle

Traitement des exceptions (gestion des erreurs)

-- *Terminaison du bloc , obligatoire*

END ;

/

Type de blocs

- Bloc anonyme
- Procédure
- Fonction

Type de blocs

Bloc anonyme

- Structure classique (1 à 3 sections)
- Un bloc ne peut être vide. Il doit contenir une instruction (il peut donc contenir l'instruction NULL)

[**DECLARE**]

BEGIN

Instructions

[**EXCEPTION**]

END;

/

Type de blocs

Bloc anonyme – Exemple 1

```
DECLARE x INTEGER;  
BEGIN  
  x := 1;  
END;
```

Type de blocs

Bloc anonyme – Exemple 2

```
DECLARE  
    var_x          VARCHAR2(5).  
BEGIN  
    SELECT nom_colonne  
    INTO    var_x  
    FROM    nom_table  
EXCEPTION  
    WHEN nom_exception THEN  
        ...  
END ;  
/
```

Type de blocs

Procédure

- Bloc PL/SQL nommé
- puis compilé et stocké dans la base

PROCEDURE nom IS

BEGIN

Instructions

[**EXCEPTION**]

END;

/

Type de blocs

Procédure – Exemple

```

PROCEDURE   proc_exemple           IS
    var_x    VARCHAR2( 5 );
BEGIN
    SELECT   nom_colonne
    INTO     var_x
    FROM     nom_table
EXCEPTION
    WHEN nom_exception THEN
        ...
END ;
/

```

EXCEPTION

```

WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée');
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);

```

→ Commande SQL qui

- Crée la procédure PL/SQL
- compile et stocke dans la base le bloc PL/SQL compris entre le BEGIN et le END, référencé par 'proc_exemple'

Type de blocs

Procédure

Remarques :

- Exécution (auto) de la procédure :

SQL> EXECUTE proc_exemple

- **Il ne faut pas mettre d'instructions de fin de transaction (COMMIT, ROLLBACK) ni d'ordres DDL dans une procédure.**
La décision d'enregistrer (COMMIT) ou d'annuler (ROLLBACK) la transaction doit être prise par le **programme appelant**, pas par la procédure elle-même.

Les principaux ordres DDL

Commande	Description	Exemple
CREATE	Créer une table, une base, un index	CREATE TABLE etudiants (id INT, nom TEXT);
ALTER	Modifier une table existante	ALTER TABLE étudiants ADD COLUMN age INT;
DROP	Supprimer complètement une table	DROP TABLE etudiants;
TRUNCATE	Vider une table (supprimer toutes les lignes)	TRUNCATE TABLE etudiants;
RENAME	Renommer une table	RENAME TABLE etudiants TO élèves;

```
CREATE TABLE comptes (  
    id NUMBER PRIMARY KEY,  
    nom VARCHAR2(50),  
    solde NUMBER  
);
```

❌ MAUVAISE PRATIQUE (COMMIT dans la procédure)

sql

```
CREATE PROCEDURE transfert(  
    p_source NUMBER, p_dest NUMBER, p_montant NUMBER  
) AS  
BEGIN  
    UPDATE comptes SET solde = solde - p_montant WHERE id = p_source;  
    UPDATE comptes SET solde = solde + p_montant WHERE id = p_dest;  
    COMMIT; -- ← DÉCISION DANS LA PROCÉDURE (à éviter)  
END;  
/
```

```

CREATE PROCEDURE transfert(
    p_source NUMBER, p_dest NUMBER, p_montant NUMBER
) AS
BEGIN
    UPDATE comptes SET solde = solde - p_montant WHERE id = p_source;
    UPDATE comptes SET solde = solde + p_montant WHERE id = p_dest;
    -- PAS DE COMMIT NI ROLLBACK ici !

END;
/

```

Programme appelant (qui décide)

```

BEGIN
    SAVEPOINT sp;

    transfert(1, 2, 100); -- Appel de la procédure

    -- Vérification avant décision
    IF (SELECT solde FROM comptes WHERE id = 1) >= 0 THEN
        COMMIT; -- ✅ Décision ici !
        DBMS_OUTPUT.PUT_LINE('Transfert validé');
    ELSE
        ROLLBACK TO sp; -- ✅ Ou annulation ici
        DBMS_OUTPUT.PUT_LINE('Transfert annulé (solde insuffisant)');
    END IF;
END;
/

```

Type de blocs

Fonction

Fonction : procédure retournant une valeur

```
FUNCTION    nom RETURN    type_données IS
BEGIN
    Instructions
RETURN valeur;
[ EXCEPTION ]
END;
/
```

Type de blocs

Fonction – exemple

```
SQL> CREATE OR REPLACE FUNCTION solde (no INTEGER)  
RETURN REAL IS le_solde REAL;  
BEGIN  
SELECT solde INTO le_solde FROM clients  
WHERE no_cli = no;  
RETURN le_solde;  
END;  
/
```

Type de blocs

Fonction

Remarques :

- Utilisation de fonction au sein d'une requête SQL

Exemple :

```
SQL> SELECT solde(1000) FROM dual ;  
Solde(1000)  
-----  
12024,50
```

- Appel d'une fonction comme une procédure *provoque une erreur*

Exemple : fonction mdp(INTEGER)

```
SQL> execute mdp(2);  
BEGIN mdp(2); END;  
*  
ERREUR a la ligne 1 :  
ORA-06550: line 1, column 7:  
PLS-00221: 'MDP' is not a procedure or is undefined  
ORA-06550: line 1, column 7:
```

Types de variables

- Variables locales

- Constantes
- Composites
- Références

```
DECLARE
```

```
-- Toutes ces variables sont LOCALES au bloc
```

```
v_salaire NUMBER;           -- Variable simple
```

```
v_pi CONSTANT NUMBER := 3.14; -- Constante
```

```
v_employe employes%ROWTYPE; -- Variable composite (référence)
```

```
BEGIN
```

```
-- Utilisation dans le bloc
```

```
END;
```

- Variables de l'environnement extérieur à PL/SQL

- Attachées (Bind)
- Hôtes (Host)

```
-- Variables qui existent en dehors du bloc PL/SQL
```

```
-- mais qu'on peut utiliser à l'intérieur
```

Déclaration des variables en PL/SQL

- Syntaxe

Identificateur [CONSTANT] type _données [**NOT NULL**]
[:= expression];

- Exemple

DECLARE

v_datenaissance
v_departement
v_ville
c_cumul

DATE;
NUMBER(2) NOT NULL := 10;
VARCHAR2(13) := 'Paris';
CONSTANT NUMBER := 1000;

Assignation des variables

- Syntaxe

Identificateur := expr ;

- Exemples

- Affecter la date de naissance du fils d'un employés

v_datenaissance := '23-SEP-2004' ;

- Fixer le nom d'un employé à 'Clément'

v_nom := 'Clément' ;

Types de données scalaires

Types de base – chaînes de caractères

- VARCHAR2(n)/NVARCHAR2(n) : Chaîne de n caractères ($n < 4000\text{octets}$) – longueur variable
(préfixe N : prise en compte de la localisation – *NLS/National Language Support*)
- CHAR(n)/NCHAR(n) : Chaîne de n caractères ($n < 2000\text{octets}$) – longueur fixe, rempli par des espaces
Char → ASCII. 2000 oct/char
NCHAR → UNICODE 1000 oct/char
- LONG/RAW/LONG RAW : Chaîne de caractère ou de données binaires (*raw*) de longueur variable ($< 2000\text{octets}$)

Types de données scalaires

Types de base – nombres et date

- NUMBER (*n* , *m*) : Réel avec *n* chiffres significatifs et *m* décimaux
- INTERGER/FLOAT : Nombre entier/réel signé
- BINARY_INTEGER : Nombre entier signé sur 32 bits (utilisation de la librairie mathématique)
- DATE : date entre le 1 janvier -4712 et le 21 décembre 9999
- BOOLEAN : booléen

NB : pas d'équivalent en SQL.

Il n'est donc pas possible

- de stocker un booléen dans une table
- de sélectionner un élément de table dans un variable booléenne
- d'utiliser un booléen dans une requête SQL ou une fonction PL/SQL appelée dans une requête SQL (paramètre en entrée ou valeur de retour)

Déclaration des variables scalaires

Exemples

v_job	VARCHAR2(9).
v_count	BINARY_INTEGER := 0;
v_sal_total	NUMBER(9,2).
v_date	DATE := SYSDATE +7;
v_tauxtaxe	CONSTANT NUMBER(3,2) := 8.25;
v_valide	BOOLEAN NOT NULL := TRUE;

Attribut %TYPE

Définition : déclaration d'une variable associée à

- Une colonne d'une table dans la BD
- Une variable précédemment définie

Exemples :

v_nom	emp.nom%TYPE;
v_sal_annuel	NUMBER (7,2);
v_sal_mensuel	v_sal_annuel %TYPE := 2000;

Types de données composites

Types :

- Les tables PL/SQL
- Les records PL/SQL

(voir plus loin)

Variables attachées/*Bind variables*

- Variables de substitution dans les requêtes SQL
Variable référençant des variables non-PL/SQL
- Economie d'analyse de la requête, donc gain de temps pour Oracle
→ Mécanisme considéré par certains comme le point clé de performance d'Oracle



Variables attachées/*Bind variables*

Déclaration

Déclaration, en SQL*Plus, d'une variable attachée : utilisation de la commande VARIABLE

Exemple :

```
VARIABLE nom _variable type _variable ;
```

```
VARIABLE g_sal_mensuel NUMBER(7,2);
```

Variables attachées/*Bind variables*

Référence aux variables non-PL/SQL

Exemple :

- Stocker le salaire mensuel dans une variable globale SQL*Plus

`:g_sal_mensuel := v_sal_annuel / 12;`

- Faire référence aux variables non-PL/SQL comme des variables hôte
- Précéder la référence par un ' : '

Variables attachées/*Bind variables*

Exemple d'utilisation – optimisation

Exemple de requête SELECT générées des milliers de fois :

```
SELECT fname , lname , pcode FROM cust WHERE id = 674;  
SELECT fname , lname , pcode FROM cust WHERE id = 234;  
SELECT fname , lname , pcode FROM cust WHERE id = 332;  
...
```

A chaque soumission d'une requête,

- Vérification si la requête a déjà été soumise
- Si oui, récupération du plan d'exécution de la requête, et exécution de la requête

Variables attachées/*Bind variables*

Exemple d'utilisation – optimisation

- Si non,
 - analyse syntaxique de la requête
 - définition des différentes possibilités d'exécution
 - définition du plan d'exécution optimal→ Processus coûteux en temps CPU, alors que seule la valeur de `id` change !
- Solution : réutiliser le plan d'exécution existant
→ Nécessite d'utiliser des *variables attachées* :
 - Substitution de la valeur par la variable attachée
 - Envoi de la même requête pour toutes les valeurs de `id`
 - Exemple :
SELECT fname , lname , pcode **FROM** cust **WHERE** id = :cust_no ;

Bloc PL/SQL

syntaxe et directives

- Les instructions peuvent être écrites sur plusieurs lignes.
Les unités lexicales peuvent être séparées par des espaces :
 - Délimiteurs
 - Identificateurs
 - Littéraux (ou constantes)
 - Commentaires

Bloc PL/SQL

syntaxe et directives

Les littéraux

- Les dates et les chaînes de caractères délimitées par deux simples cotes

```
V_nom := 'Thierry';  
v_annee := to_number(to_char(v_DateFinPeriode, 'YY'));
```

- Les nombres peuvent être des valeurs simples ou des expressions

```
V_salaire := 1500 + 300 ;
```

Commentaires dans le code

- Précéder un commentaire écrit sur une seule ligne par '--'.
- Placer un commentaire écrit sur plusieurs lignes entre les symboles '/*' et '*/'.
- Exemple :

```
v_sal    NUMBER(9,2);  
BEGIN  
    /* Ceci est un commentaire qui peut  
       être écrit sur plusieurs lignes */  
END; -- Ceci est un commentaire sur une ligne
```

Les Fonctions SQL en PL/SQL

- Les fonctions sur les nombres
- Les fonctions sur les chaînes de caractères
- Les fonctions de conversion de type de données
- Les fonctions de dates

Exemples de fonctions SQL en PL/SQL

Exemples :

- Recomposer l'adresse d'un employé :

```
V_Adr Complete  : = V_Rue || CHR(32) || V_Ville || CHR(32) ||  
                   V_CodePostal ;
```

- Convertir le nom en majuscule

```
V-Nom := UPPER (V-Nom) ;
```

- Extraction d'une partie de la chaîne

```
V-chr := Substr ( 'PL/SQL' ,4,3) ;
```

- Remplacement d'une chaîne par une autre

```
V-chr := Replace ( 'Serv1/ Prod/ tb-client' , 'Prod' , 'Valid' ) ;
```

REPLACE(chaîne, recherche, remplacement) : fonction qui remplace toutes les occurrences

Blocs imbriqués et portée des variables

```

...
x      BINARY_INTEGER;
BEGIN -- Debut de la portee de x

...
DECLARE
    y      NUMBER;
    BEGIN -- debut de la porte de y

        ...
        END; -- fin de la portee de y

    ...
END; -- fin de la portee de x

```

DECLARE

x BINARY_INTEGER := 10; -- Variable x (visible partout)

BEGIN

DBMS_OUTPUT.PUT_LINE('Bloc principal - x = ' || x); --  OK

DECLARE

y NUMBER := 20; -- Variable y (visible seulement ici)

BEGIN

DBMS_OUTPUT.PUT_LINE('Sous-bloc - x = ' || x); --  OK (x hérité)

DBMS_OUTPUT.PUT_LINE('Sous-bloc - y = ' || y); --  OK

x := 30; -- Modification de x (visible partout)

END; -- ← y meurt ici !

DBMS_OUTPUT.PUT_LINE('Retour bloc principal - x = ' || x); --  OK (30)

-- DBMS_OUTPUT.PUT_LINE(y); --  ERREUR : y n'existe plus !

END; -- ← x meurt ici !

Opérateurs dans PL/SQL

- Identique à SQL
 - Logique
 - Arithmétique
 - Concaténation
 - Parenthèses pour contrôler l'ordre des opérations
- Opérateur supplémentaire : Opérateur d'exponentiation **

Utilisation des variables liées

- Pour référencer une variable en PL/SQL, on doit préfixer son nom par un ':'
- Exemple :

```
:code_retour := 0;  
IF vérifier_crédit_ok (compt_no) THEN  
    :code_retour := 1;  
END IF;
```

Exemple concret

Étape 1 : Créer une variable dans SQL*Plus

```
sql
```

```
-- Variable créée DANS L'OUTIL (SQL*Plus), pas dans PL/SQL  
VARIABLE code_retour NUMBER;
```

Étape 2 : L'utiliser dans un bloc PL/SQL

```
sql
```

```
BEGIN  
    :code_retour := 0; -- Le : signifie "variable externe"  
  
    IF verifier_credit_ok(compt_no) THEN  
        :code_retour := 1; -- Même syntaxe  
    END IF;  
END;  
/
```

Étape 3 : Voir le résultat

```
sql
```

```
PRINT code_retour; -- Affiche la valeur
```

Instructions SQL dans PL/SQL

- Extraire une ligne de données à partir de la BD par la commande SELECT. Un seul ensemble de valeurs peut être retourné
- Effectuer des changements aux lignes dans la BD par les commandes du LMD
- Contrôler des transactions avec les commandes COMMIT, ROLLBACK et SAVEPOINT
- Déterminer les résultats du LMD avec des curseurs implicites (voir plus loin)

Instruction SELECT dans PL/SQL

- Récupérer une donnée de la BD avec SELECT.
- Syntaxe

```
SELECT          liste_sélection
INTO            { nom_var [, nom_var] ...
                  | nom_record }
FROM            table
WHERE           condition ;
```

Instruction SELECT dans PL/SQL

- La clause INTO est obligatoire
- Exemple

DECLARE

v_deptno
v_loc

NUMBER(2);
VARCHAR2(15);

BEGIN

SELECT
INTO
FROM
WHERE

deptno , loc
v_deptno , v_loc
dept
nom_d = 'INFORMATIQUE';

...
END;

Instruction SELECT dans PL/SQL

- Retourne la somme des salaires de tous les employés d'un département donné.
- Exemple

DECLARE

v_som_sal
v_deptno

emp.sal%TYPE;
NUMBER NOT NULL := 10;

BEGIN

SELECT
INTO
FROM
WHERE

sum (sal) *--fonction d'agr*
v_som_sal
emp
deptno = v_deptno ;

END;

Manipulation de données en PL/SQL

- Effectuer des mises à jour des tables de la BD utilisant les commandes du LMD :
 - INSERT
 - UPDATE
 - DELETE

Insertion de données

- Ajouter les informations d'un nouvel employé à la table emp
- Exemple

```
DECLARE
    v_empno          NUMBER NOT NULL := 105;
BEGIN
    INSERT INTO emp (empno, emp_nom, poste, deptno)
    VALUES (v_empno, 'Clément', 'Directeur', 10);
END;
```

Mise à jour de données

- Augmenter le salaire de tous les employés dans la table emp qui ont le poste d'enseignant.
- Exemple

DECLARE

 v_augm_sal emp.sal%TYPE := 2000;

BEGIN

UPDATE

SET

WHERE

emp

sal := sal + v_augm_sal

job = 'Enseignant';

END; 1

Suppression de données

- Suppression des lignes appartenant au département 10 de la table emp
- Exemple

DECLARE

 v_ deptno emp.deptno%TYPE := 10;

BEGIN

DELETE FROM emp
 WHERE deptno = v_deptno ;

END;

Structure de contrôle dans PL/SQL

- IF conditionnel :

```
IF      THEN      END IF ;  
IF      THEN      ELSE      END IF ;  
IF      THEN      ELSIF      END IF ;
```

- Les boucles :

```
LOOP      END LOOP;  
FOR      LOOP      END LOOP;  
WHILE      LOOP      END LOOP;
```

Instruction IF

Syntaxe

```
IF condition THEN
    énoncés ;
[ELIF condition THEN
    énoncés ;]
[ELSE
    énoncés ;]
END IF ;
```

Instruction IF

Exemple de IF simple

Mettre le ID de l'employé 'MARK' à 101.

```
IF v_nom = 'MARK' THEN  
    v_ID := 101;  
END IF;
```

IF simple

- Si le nom de l'employé est 'CLEMENT', lui attribuer le poste 'Enseignant', le département n° 102 et une commission de 25 % sur son salaire actuel
- Exemple

```
...  
IF v_nom = 'Clément' THEN  
  v_poste := 'Enseignant';  
  v_deptno := 102;  
  v_nouv_comm := sal * 0.25;  
END IF;  
...
```

IF-THEN-ELSE

- Si le nom de l'employé est 'CLEMENT', lui attribuer le poste 'Enseignant', le département n° 102 et une commission de 25 % sur son salaire actuel, sinon afficher le message 'Employé inexistant'
- Exemple

```
...  
IF v_nom = 'Clément' THEN  
    v_poste := 'Enseignant';  
    v_deptno := 102;  
    v_nouv_comm := sal * 0.25;  
ELSE  
    DBMS_OUTPUT.PUT_LINE('Employé inexistant');  
END IF;  
...
```

IF-THEN-ELSIF

- Pour une valeur donnée en entrée, retourner une valeur calculée
- Exemple

```
...  
IF          v_debut > 100  THEN      RETURN ( 2 * v_debut );  
ELSIF      v_debut >= 50  THEN      RETURN ( 5 * v_debut );  
ELSE       RETURN ( 1 * v_debut );  
END IF ;  
...
```

CASE

Exemple :

```
select id_bcr_expo ,id_gest_fin ,rownum ,  
       case when rownum <= 50 then '1-50 rownum ' || rownum  
         when rownum <= 100 then '51-100 rownum ' || rownum  
       else 'plus de 100 ' end  
from bcr_i.tb_gst_financement  
  
where rownum <= 100
```

Cette requête permet de retourner un contenu dans une colonne en fonction d'une condition

Boucle de base

Syntaxe

```
LOOP                                     -- d'initialisation
    énoncé 1;                           -- énoncé
    ...
    EXIT [ WHEN condition ];            -- énoncé EXIT
END LOOP;
```

When : condition est une variable booléenne ou expression (TRUE, FALSE, ou NULL);

Boucle de base

Exemple

Insérer 10 articles avec la date d'aujourd'hui.

```
...  
    v_Date          DATE;  
    v_compteur      NUMBER(2) := 1;  
BEGIN  
...  
    v_Date := SYSDATE;  
    LOOP  
        INSERT INTO article (Artno , ADate)  
            VALUES (v_compteur , v_Date);  
        v_compteur := v_compteur + 1;  
        EXIT WHEN v_compteur > 10;  
    END LOOP;  
...
```

Boucle FOR

Syntaxe

```
FOR indice IN [REVERSE]
    borne_inf .. Borne_sup LOOP
    énoncé 1;
    énoncé 2;
    .....
END LOOP;
```

- Utiliser la boucle FOR pour raccourcir le test d'un nombre d'itérations
- Ne pas déclarer l'indice, il est déclaré implicitement

Boucle FOR

exemple

Insérer Nb articles indexés de 1 à Nb avec la date du système en utilisant la boucle FOR

```
ACCEPT Nb PROMPT 'Donner le nombre article :'
```

```
...  
    v_Date  DATE;  
BEGIN  
...  
    v_Date := SYSDATE;  
    FOR i IN 1 .. &Nb LOOP  
        INSERT INTO article (Artno , ADate)  
            VALUES (i, v_Date);  
    END LOOP;  
...  
END
```

Boucle WHILE

Syntaxe

```
WHILE condition LOOP  
    énoncé 1;  
    énoncé 2;  
    .....  
END LOOP;
```

- La condition est évaluée au début de chaque itération
- Utiliser la boucle WHILE pour répéter des énoncés tant que la condition est vraie

Boucle WHILE

Exemple

```
ACCEPT p_itemtot PROMPT 'Donner le total max de l''achat  
d''un article'
```

```
DECLARE
```

```
    v_Date                                DATE;
```

```
    v_compteur                            NUMBER(2) := 1;
```

```
BEGIN
```

```
...
```

```
    v_Date := SYSDATE;
```

```
    WHILE v_compteur <= &p_itemtot LOOP
```

```
        INSERT INTO article (Artno , ADate)
```

```
            VALUES (v_compteur , v_Date);
```

```
        v_compteur := v_compteur + 1;
```

```
    END LOOP;
```